



FINTRACKER

Primera Iteración de Resolución del Problema

Mateo Pérez Suarez (79439806Z)
Íñigo Peña de las Heras (20983346D)

Repositorio del Proyecto

<https://github.com/MateoPerezSuarez/FinTracker>

Los contenidos relacionados a esta entrega se encuentran en `src/Primera_Resolucion`.

28 de noviembre de 2025

Índice

1. Revisión y Optimización de la Entrega Previa	2
2. Shallow ML	2
2.1. Text Classification	2
2.1.1. Representación del Lenguaje Utilizada	2
2.1.2. Modelos y Resultados	3
3. Deep Learning Approach	3
3.1. Text Classification	3
3.1.1. Representacion secuencial con Word2Vec	3
3.1.2. Arquitecturas	4
3.2. Summarization(Generación de titulares): src/Primera_Resolucion/Summarization	5
3.2.1. Arquitectura del Modelo y Mecanismo de Atención	5
3.2.2. Resultados y comparativa: src/Primera_Resolucion	6
3.2.3. Métricas de Evaluación: ROUGE	6
3.2.4. Análisis Cuantitativo	6
3.2.5. Análisis Cualitativo y Errores de Generación	7
3.2.6. Conclusión de la Iteración	7

1 Revisión y Optimización de la Entrega Previa

Partiendo desde el punto en el que lo dejamos en la entrega anterior, el primer paso que hemos llevado a cabo ha sido revisar y reforzar el trabajo previamente realizado, incorporando las observaciones que se nos sugirieron. En la revisión detectamos algunos fallos conceptuales importantes, especialmente relacionados con el uso inadecuado de ciertos procesos de preprocesamiento y de la arquitectura BERT.

En particular teníamos dos errores principales:

- **Uso incorrecto del token [CLS] en BERT:** En la entrega anterior utilizamos el embedding asociado al token [CLS] de forma inapropiada, empleándolo como sustituto de las embeddings contextuales completas. Sin embargo, el token [CLS] está diseñado para actuar como representación global de la secuencia únicamente en tareas de clasificación, mientras que para tareas de análisis palabra a palabra o extracción de información debe preservarse la matriz completa de embeddings contextualizados.
- **Lematización aplicada indebidamente a embeddings no contextuales:** Observamos que, además de aplicarla a modelos tradicionales como Bag-of-Words y TF-IDF (donde sí es recomendable), habíamos aplicado lematización también a modelos no contextuales como Word2Vec y FastText. Este enfoque es incorrecto, ya que la lematización elimina variación morfológica que estos modelos necesitan para aprender adecuadamente las relaciones semánticas basadas en coocurrencias reales del corpus.

Estos errores no solo afectaban a la calidad de las embeddings, si no que también podía perjudicar el rendimiento de los mismos para las tareas que queremos llevar a cabo en esta entrega. Por ello, hemos decidido solucionarlo lo primero, corregir estos fallos y ajustar el pipeline con el fin de obtener embeddings de mejor calidad para cada tipo de modelo y garantizar así una base más sólida para las tareas de clasificación y summarization vamos a abordar a continuación.

2 Shallow ML

2.1 Text Classification

El objetivo de esta aquí es establecer una línea base sólida antes de introducir modelos más complejos de Deep Learning, evaluando hasta qué punto representaciones clásicas del texto y clasificadores lineales pueden resolver eficazmente la tarea. Para ello, utilizo representaciones TF-IDF y modelos como Multiclass Logistic Regression y Linear SVM, dos enfoques ampliamente contrastados en Procesamiento del Lenguaje Natural cuando trabajamos con vectores de alta dimensionalidad y datos dispersos.

2.1.1. Representación del Lenguaje Utilizada

- **TF-IDF:** se cargan 5 000 term features de la matriz TF-IDF generada previamente. Cada noticia queda en un vector disperso donde cada peso refleja la importancia relativa del termino en el documento. Se divide el dataset en train/validacion/test con proporción 80/10/10 para evaluar de forma comparable los distintos modelos.
- **Word2Vec:** se carga el modelo propio entrenado sobre el corpus financiero (16 863 terminos, 300 dimensiones). Cada texto se tokeniza, se toman los vectores de las palabras presentes y se promedia para obtener un unico vector denso de 300 dimensiones por documento. Se usa el mismo split 80/10/10 para alinearlos con TF-IDF.

2.1.2. Modelos y Resultados

■ Multiclass Logistic Regression:

- **Configuracion:** solver lbfgs (estable para multiclase), regularizacion L2 implicita, max_iter=1000 para asegurar convergencia en el espacio de 5 000 dimensiones. Es el baseline clasico para texto linealmente separable con pesos interpretables.
- **Con TF-IDF:** val acc 0.9050, val macro-F1 0.8397; test acc 0.9050, test macro-F1 0.8245. El macro-F1 cercano al accuracy indica que las clases minoritarias (como podrian ser *Informal Conversational Lenguaje* o *Quantum Computing and Military Technology* en nuestro caso.) no se quedan muy atras; obteniendo un modelo estable y rapido de entrenar.
- **Subsampling (TF-IDF):** para balancear el train recortando clases mayoritarias para probar mejora en minoritarias. Los resultados empeoran porque se pierde variabilidad util y el modelo generaliza peor; por lo que descartamos la estrategia.
- **Con Word2Vec** val acc 0.8198, val macro-F1 0.7348; test acc 0.8236, test macro-F1 0.7085. Word2Vec puede mostrar un comportamiento distinto en las clases minoritarias debido a su capacidad de capturar similitudes semánticas.

TF-IDF presenta mejores resultados, lo cual indica que los términos específicos y su peso discriminativo son más importantes que la información semántica contextual.

■ SVM lineal (LinearSVC):

- **Configuracion:** hinge loss con C=1.0. Se elige SVM porque maximiza el margen en espacios de alta dimension, robusto a sobreajuste con datos dispersos y con buena capacidad para clases desbalanceadas cuando se optimiza el margen.
- **Con TF-IDF:** val acc 0.9225, val macro-F1 0.8974; test acc 0.9283, test macro-F1 0.8716. El espacio de 5 000 dimensiones vuelve a las clases casi separables linealmente; la SVM extrae un hiperplano con margen amplio y domina en shallow.
- **Con Word2Vec:** val acc 0.8450, val macro-F1 0.8117; test acc 0.8566, test macro-F1 0.8027. Mejora frente a LogReg con los mismos vectores gracias al margen maximo, pero sigue por debajo de TF-IDF.

TF-IDF + SVM usa terminos especificos ponderados y un clasificador que aprovecha la separabilidad lineal en alta dimension, dando los mejores resultados para la clasificación de los textos. Word2Vec promedio es mas compacto y semantico pero pierde detalle lexical util; solo mejora ligeramente con SVM respecto a LogReg. El subsampling no ayuda por la perdida de ejemplos de clases dominantes, así que conservamos el train completo.

3 Deep Learning Approach

3.1 Text Classification

3.1.1. Representacion secuencial con Word2Vec

- **Embeddings propios:** Reutilizamos el modelo financiero pero ahora conservando el orden de las palabras. Cada texto se tokeniza y se rellena (padding) o se limita hasta MAX_LEN=250 para equilibrar cobertura de palabras y coste de computo. Construimos un tensor (num_textos, 250, 300) con vectores por posición. Codificamos las etiquetas con LabelEncoder y se usa un split

estratificado 80/20 para train/test. Esta configuración mantiene el contexto local y distribuye la misma semántica vista en shallow pero sin promediarla.

- **Embeddings preentrenados de Google News:** Para comparar resultados, vamos a utilizar un modelo Word2Vec preentrenado por Google en 3 millones de noticias de Google News. Este modelo contiene vectores de 300 dimensiones para 3 millones de palabras y frases, entrenado con un corpus masivo y diverso. Generamos la misma matriz secuencial (250 pasos, 300 dims) para comparar cobertura generalista frente a especialización de dominio.

De esta manera comprobaremos si el vocabulario especializado en finanzas es más útil que el conocimiento general, o si por el contrario la mayor cobertura y calidad de los embeddings preentrenados compensa la falta de especialización en el dominio financiero.

3.1.2. Arquitecturas

- **Convolutional Neural Network**

Para abordar la tarea de clasificación de texto, se ha diseñado e implementado una arquitectura basada en Redes Neuronales Convolucionales unidimensionales (1D-CNN). Aunque originalmente concebidas para visión artificial, las CNNs han demostrado una gran eficacia en el Procesamiento del Lenguaje Natural (PLN) gracias a su capacidad para capturar patrones locales y dependencias espaciales entre palabras adyacentes.

La arquitectura del modelo se compone de un flujo secuencial de capas diseñado para transformar los vectores de entrada en probabilidades de clase, estructurado en tres bloques principales: extracción de características, reducción de dimensionalidad y clasificación.

- **Bloque Convolutivo:** La entrada del modelo es procesada inicialmente por una capa Conv1D (Convolución Unidimensional), en la que se han definido 128 filtros y un tamaño kernel de 5. Esta capa actúa como el principal extractor de características del sistema
- **Reducción de Dimensionalidad y Selección:** Tras la convolución, se aplica una capa de GlobalMaxPooling1D. Esta operación examina cada uno de los 128 mapas de características generados anteriormente a lo largo de toda la secuencia temporal y selecciona el valor máximo de cada uno.
- **Clasificación y Regularización:** El vector de características resultante (de longitud 128) se introduce en un perceptrón multilayer para la decisión final. Primero por una *Fully Connected Dense Layer* (64 neuronas) responsable de combinar no-linealmente las características extraídas por la convolución; Incluyendo una capa de dropout del 5%; y por último por la capa de salida, una dense layer de 5 neuronas con función de activación Softmax proyecta los resultados en una distribución de probabilidad.
- **Entrenamiento:** Como loss function utilizamos la *sparse_categorical_crossentropy*. Para el optimizer utilizamos Adam optimizer, el estándar actual en deep learning.

- **Bi-LSTM**

Para complementar el análisis y capturar dependencias secuenciales a largo plazo, hemos implementado una segunda arquitectura basada en Redes Neuronales Recurrentes, específicamente una *Bidirectional Long Short-Term Memory* (Bi-LSTM). Este modelo está diseñado para procesar el texto considerando tanto el contexto pasado como el futuro de cada palabra.

La arquitectura sigue un flujo de procesamiento secuencial avanzado, utilizando doble pooling antes de la clasificación:

- **Procesamiento Secuencial Bidireccional:** El núcleo del modelo es una capa Bi-LSTM con 128 unidades por dirección. A diferencia de las redes estándar, esta capa procesa la secuencia de entrada en ambos sentidos, generando una representación contextual rica de 256 dimensiones (128 de ida + 128 de vuelta) para cada palabra, manteniendo la secuencia completa gracias al parámetro *return_sequences=True*.
- **Estrategia de Pooling Dual:** Implementamos un mecanismo híbrido de reducción de dimensionalidad para sintetizar la información. Aplicamos en paralelo un *GlobalAveragePooling1D* (para capturar el contexto general o "promedio" de la frase) y un *GlobalMaxPooling1D* (para detectar las características más destacadas o señales fuertes). Conectamos estos vectores, dándole a la red una visión más completa del texto.
- **Clasificación y Regularización:** Primero pasa por una dense layer de 64 neuronas con activación ReLU; seguida de una capa de Dropout del 50 % (0.5); y finalmente, la capa de salida de 5 neuronas con activación Softmax para la clasificación multiclase.
- **Entrenamiento:** Mantenemos la consistencia con el modelo anterior utilizando *sparse_categorical_crossentropy* como función de pérdida, y el optimizador Adam con un ratio de aprendizaje (*learning rate*) de 0.001.

3.2 Summarization(Generación de titulares): src/Primera_Resolucion/Summarization

En esta primera iteración para la resolución de la tarea de generación automática de titulares (headline generation), hemos optado por implementar una arquitectura basada en modelos de Seq2Seq. Si bien el state of art actual tiende más hacia arquitecturas puramente basadas en Transformers, hemos decidido fundamentar nuestra aproximación inicial en RNNs con mecanismos de atención como primera iteración, para comprender con mayor profundidad el flujo de información y establecer una base más sólida antes de iterar hacia modelos generativos más complejos. Para esto, a continuación, detallamos la arquitectura que hemos propuesto, la estrategia de representación vectorial y la metodología experimental que hemos decidido seguir.

3.2.1. Arquitectura del Modelo y Mecanismo de Atención

Para abordar el problema de resumir artículos financieros extensos en titulares concisos, hemos diseñado una arquitectura Encoder-Decoder potenciada por un mecanismo de atención. El elegir estos componentes específicos viene dado por intentar buscar un equilibrio entre eficiencia computacional y capacidad de captura de dependencias a largo plazo.

En primer lugar, para el componente recurrente, hemos seleccionado unidades GRU (Gated Recurrent Units) en lugar de LSTM. Esto es porque queríamos intantar aprovechar más la eficiencia de las GRU, que por lo aprendido tienen menos gates (GRU tiene 2 (update, reset) vs 3 de LSTM (input, forget, output)) y parámetros, que nos puede servir para reducir el coste computacional y el riesgo de overfittear en datasets de tamaño medio como el nuestro, manteniendo un rendimiento comparable a las LSTM en tareas de modelado de secuencias.

El Encoder lo hemos configurado como bidireccional. Esto consideramos que es relevante en el dominio de nuestro proyecto (el financiero), ya que el contexto necesario para determinar la relevancia de una entidad o cifra a menudo depende tanto de los términos precedentes como de los posteriores. Al procesar la secuencia en ambas direcciones, generamos una hidden representation que encapsula el contexto global de cada token.

Una de las limitaciones que podríamos tener, viene dada por el modelo Seq2Seq, que podría darse un cuello de botella que se produce al intentar condensar toda la información de un artículo, pongamos de

512 tokens en un único vector de contexto fijo. Para mitigar esto, hemos implementado el Mecanismo de Atención de Bahdanau (O atención Aditiva, como se menciona en la teoría). A diferencia de otras atenciones como la atención multiplicativa (Luong), la atención de Bahdanau aprende una función de alineación mediante una red neuronal feed-forward que calcula scores de relevancia entre el hidden state actual del decoder y todos los hidden states del encoder. Esto permite al modelo "enfocar" dinámicamente su atención en diferentes segmentos del artículo que le pasemos como fuente (por ejemplo, cifras de beneficios o nombres de empresas) en cada paso de la generación del titular, lo cual creemos que mejorará sustancialmente la capacidad del modelo para sintetizar secuencias largas.

3.2.2. Resultados y comparativa: src/Primera_Resolucion

Para validar las hipótesis planteadas en nuestra metodología y determinar la eficacia de las distintas configuraciones de embeddings en la arquitectura Seq2Seq, hemos llevado a cabo una evaluación tanto cuantitativa como cualitativa.

Aunque antes de avanzar con esto, queremos comentar que hemos tenido un problema a la hora de usar BERT, ya que el tiempo de entrenamiento era demasiado largo (se nos ha apagado el portátil porque se sobrecalentaba, igual es fallo mío). Esto, como se va a ver a continuación, nos ha causado bastantes problemas a la hora de medir y comparar, por lo que hemos pensado en utilizarlos de manera más robusta para la próxima entrega, ya que consideramos que verdaderamente son los que mejores resultados nos van a dar al tratar con transformers de manera definitiva.

3.2.3. Métricas de Evaluación: ROUGE

Para la evaluación cuantitativa, hemos seleccionado el conjunto de métricas **ROUGE** (Recall-Oriented Understudy for Gisting Evaluation), que hemos visto que es el estándar *de facto* en tareas de resumen automático. Específicamente, nos hemos centrado en:

- **ROUGE-1:** Mide la superposición de unigramas (palabras individuales) entre el titular generado y el de referencia. Esta métrica es fundamental para evaluar la capacidad del modelo de capturar el **contenido informativo** clave (nombres de empresas, verbos de acción, cifras).
- **ROUGE-2:** Mide la superposición de bigramas. Es un indicador más estricto que evalúa la **fluidez** y la capacidad del modelo para construir composiciones o sintagmas más coherentes, algo crítico en el dominio financiero donde el orden de las palabras altera el significado (ej: "acciones suben" vs "suben acciones").

Hemos descartado métricas como BLEU (común en traducción) ya que ROUGE correlaciona mejor con la evaluación humana en tareas de resumen como la nuestra, donde la recuperación de información es prioritaria sobre la precisión exacta de la traducción.

3.2.4. Análisis Cuantitativo

Tras entrenar y evaluar las cuatro configuraciones (FastText Frozen/Fine-tuned y BERT Frozen/Fine-tuned) sobre el conjunto de test, obtuvimos los resultados promedio que se muestran a continuación.

Los resultados arrojaron hallazgos contraintuitivos respecto a la literatura estándar, pero reveladores sobre la complejidad del entrenamiento de modelos recurrentes con datos limitados y las restricciones que hemos sufrido:

1. **Supremacía de FastText Frozen:** Contra todo pronóstico inicial, la configuración más sencilla (**FastText Frozen**) obtuvo el mejor rendimiento con un **ROUGE-1 de 0.2229** y un **ROUGE-2**

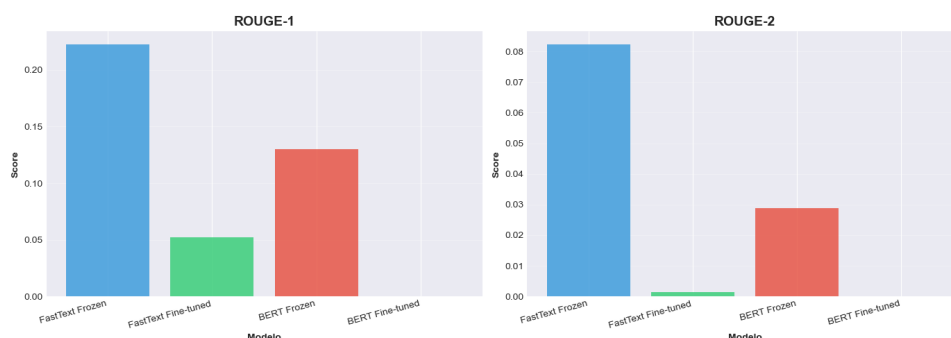


Figura 1: Comparativa de métricas ROUGE-1 y ROUGE-2 entre los cuatro modelos.

de **0.0824**. Esto sugiere que, para un dataset de tamaño medio-bajo (5000 muestras), mantener los embeddings estáticos actúa como un regularizador eficaz, permitiendo que la GRU se centre exclusivamente en aprender la estructura sintáctica sin “destruir” el espacio semántico pre-entrenado.

2. **El problema con el Fine-tuning:** Observamos una caída en los modelos con *fine-tuning*. **FastText Fine-tuned** cayó a un ROUGE-1 de 0.05. Por otro lado, **BERT Fine-tuned sale con un 0.0 absoluto**; esto se debe directamente a lo que hemos explicado arriba: al apagarse el equipo y no poder terminar el entrenamiento por falta de recursos, el modelo no ha aprendido nada útil y por eso no genera resultados válidos.
3. **BERT Frozen:** Aunque superior a las versiones fine-tuned, BERT Frozen (ROUGE-1: 0.13) no superó a FastText. Esto puede deberse a la disparidad de dimensiones (768 de BERT vs 256 de la GRU), donde la proyección necesaria para adaptar BERT al decodificador pudo haber perdido información relevante.

3.2.5. Análisis Cualitativo y Errores de Generación

Para comprender el porqué de las métricas bajas en los modelos complejos, realizamos una inspección visual de las generaciones:

- **Coherencia en Baseline:** El modelo *FastText Frozen* logró generar estructuras reconocibles relacionando entidades (ej: “doordash launches dot...”).
- **Repetición y Fallo:** Los modelos que sufrieron degradación mostraron comportamientos extraños. *FastText Fine-tuned* entró en bucles repitiendo palabras (ej: “why stock stock to to...”). En el caso de *BERT Fine-tuned*, como el entrenamiento se cortó, no generó nada con sentido, lo cual explica el resultado nulo.

3.2.6. Conclusión de la Iteración

Esta primera resolución nos permite concluir que, con los recursos que tenemos ahora mismo y usando RNNs, **lo más sencillo (FastText Frozen) ha funcionado mejor**. Intentar hacer fine-tuning a un modelo tan grande como BERT en local ha sido demasiado pesado. Para la próxima entrega, tenemos claro que el cambio a una arquitectura de Transformers puros (como BART o T5) mejorará con creces, y buscaremos una forma más eficiente de entrenarlos para que no ocurran estos problemas de hardware.


```

EJEMPLO 1

Artículo Seleccionado:
skip to navigation skip to main content skip to right column in this video: story: tesla's sales rose in several european markets

Referencia:
tesla sales rise in parts of europe amid pressure from rivals

FastText Frozen (R1: 0.000):
tech tech stocks to watch trading trading

FastText Fine-tuned (R1: 0.000):
why stock stock to to to to

BERT Frozen (R1: 0.091):
tesla tesla tesla to tesla to tesla to

BERT Fine-tuned (R1: 0.000):
...
BERT Fine-tuned (R1: 0.000):

```

Figura 2: Comparativa de resultados.

Referencias

- Bahdanau, D., Cho, K., and Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations (ICLR)*.
- Cheng, W. K., Bea, K. T., Leow, S. M. H., Chan, J. Y.-L., Hong, Z.-W., and Chen, Y.-L. (2022). A review of sentiment, semantic and event-extraction-based approaches in stock forecasting. *Mathematics*, 10(14):2437.
- Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. (2014). Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*.
- Liu, Y. and Lapata, M. (2019). Text summarization with pretrained encoders. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3730–3740. Association for Computational Linguistics.
- Rush, A. M., Chopra, S., and Weston, J. (2015). A neural attention model for abstractive sentence summarization. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 379–389. Association for Computational Linguistics.
- Sutskever, I., Vinyals, O., and Le, Q. V. (2014). Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, volume 27.
- Universidad de Deusto (2025a). Atención en rnns: Fundamentos del procesamiento del lenguaje natural. Apuntes de clase, Grado en Ciencia de Datos e Inteligencia Artificial.
- Universidad de Deusto (2025b). Fundamentos del procesamiento de lenguaje natural: Unidad 2 - técnicas de representación de palabras. Apuntes de clase, Facultad de Ingeniería. Accedido en el curso 2025-2026.
- Universidad de Deusto (2025c). Fundamentos del procesamiento de lenguaje natural: Unidad 3 - técnicas avanzadas de machine learning para nlp. Apuntes de clase, Facultad de Ingeniería. Accedido en el curso 2025-2026.